

Access Control: How systems are “protected”

Mainack Mondal

CS60112

Spring 2025



Outline

- Access control
- In practice: Unix access control
- Breaking access: memory attacks
 - Exploiting overflow
 - Return oriented programming (ROP)

Slide courtesy: Matteo Maffei, TU Wien

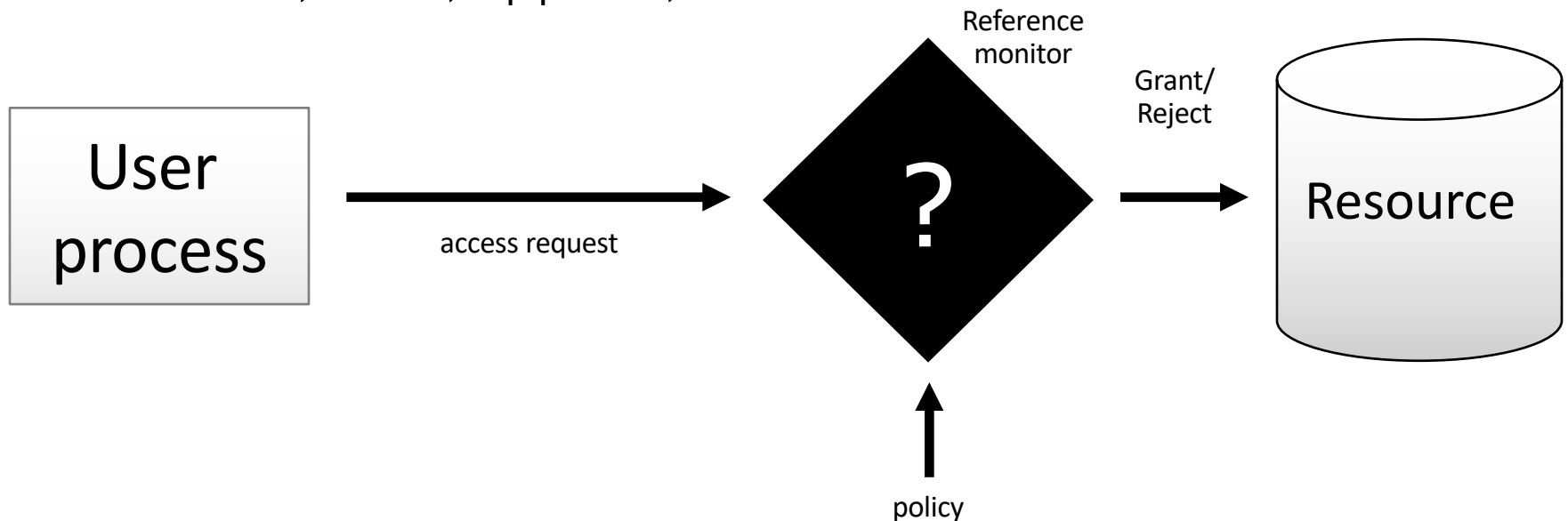
Outline

- Access control
- In practice: Unix access control
- Breaking access: memory attacks
 - Exploiting overflow
 - Return oriented programming (ROP)

Access Control

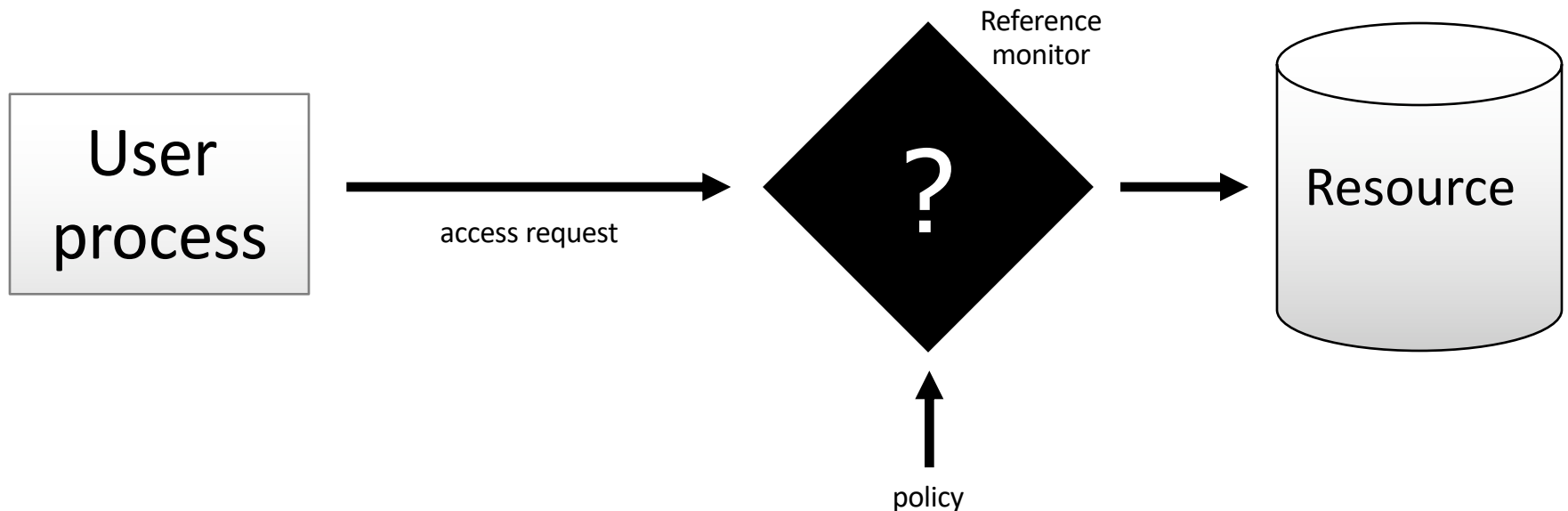
Fundamental concepts

- Principals and subjects (users, processes etc.)
 - active, access something
- Objects (files devices, storage, services, etc.)
 - passive, are accessed
- Operations
 - Observe, alter, use
 - Read, write, append, execute



Assumptions

- System knows who the user is
 - Authentication via name and password, other credential
- Access requests pass through gatekeeper
 - System must not allow monitor to be bypassed

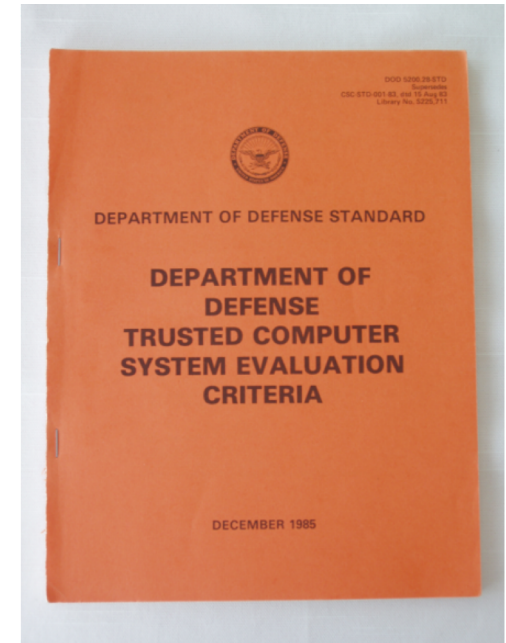


Terminology

- A **principal** is an entity that has an authenticated identity
- A **subject (or user)** is an entity that acts on behalf of a principal within a computer system
 - e.g., a process running under a user identity
- An **object** is an entity that is acted upon
 - e.g., a text file
- Observe is simply to look at the contents of an object
- Alter is to change the contents of (or add contents to) an object
- It is possible to use functionality of an object without extracting its contents

More on subjects: owner

- In computers, the owner can grant access or revoke it, and decide what type of access
 - This is called **discretionary** access control (DAC)
 - The opposite, **mandatory** access control (MAC) mostly occurs in the military / defense that handles classified data



Access control, the basic problem

- Simply listing, per subject and object, what access is allowed and/or denied is very inefficient
- We need a structured approach that is
 - efficient
 - flexible
 - testable

Access control, the basic problem

- Simply listing, per subject and object, what access is allowed and/or denied is very inefficient
- We need a structured approach that is
 - efficient
 - flexible
 - ~~testable~~

Access control matrix [Lampson]

		Objects				
		File 1	File 2	File 3	...	File n
Subjects	User 1	read	write	-	-	read
	User 2	write	write	write	-	-
	User 3	-	-	-	read	read
	...					
	User m	read	write	read	write	read

implementation concept 1: access control list

- Access control list (ACL)
 - Per object what rights the listed subjects have to the object
 - Resource identified by an index
 - Store column of matrix with the resource
 - A simple version is permission bits

	File 1	File 2	File 3	...	File n
User 1	read	write	-	-	read
User 2	write	write	write	-	-
User 3	-	-	-	read	read
...					
User m	read	write	read	write	read

implementation concept 2: capability

- Capability
 - Per subject what rights the subject has to the listed objects
 - User holds a “ticket” for each resource
- Two variations
 - store row of matrix with user, under OS control
 - unforgeable ticket in user space
- Integrity is fundamental
 - capabilities can be modified and eventually retrieved only through OS calls

	File 1	File 2	File 3	...	File n
User 1	read	write	-	-	read
User 2	write	write	write	-	-
User 3	-	-	-	read	read
...					
User m	read	write	read	write	read

Capabilities

- Capabilities serve two purposes:
 - identify the resource to be accessed
 - tell whether the operation is allowed

ACLs in simple form: permission bits

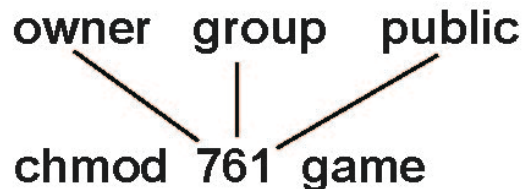
- Very simple
 - Fixed set of rights, fixed set of objects
- Very inflexible
 - Efficient to determine permission for a given object
 - No lists to search, registered directly with the objects
- Three subject types per object
 - Owner, Group, Others
- Three types of rights per subject type
 - Read, Write, eXecute

Access Lists and Groups

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

			RWX
a) owner access	7	⇒	1 1 1
			RWX
b) group access	6	⇒	1 1 0
			RWX
c) public access	1	⇒	0 0 1

- For a particular file (say *game*) or subdirectory, define an appropriate access.



Attach a group to a file

chgrp G game

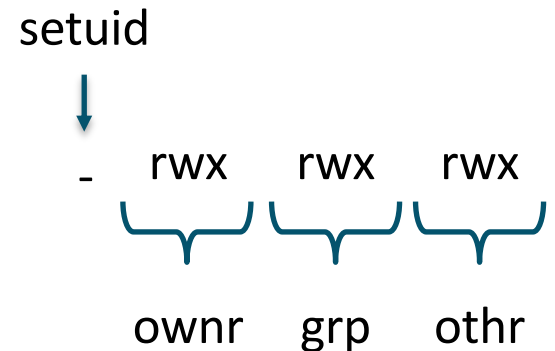
A Sample UNIX Directory Listing

-rw-rw-r--	1	pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5	pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2	pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2	pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1	pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1	pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4	pbg	faculty	512	Jul 31 10:31	lib/
drwx-----	3	pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3	pbg	staff	512	Jul 8 09:35	test/

- Question: What is the meaning of chmod 777, chmod 644, chmod 700
- Check : <http://www.filepermissions.com/articles/what-are-file-permissions-in-linux-and-mac>

Summary: Unix file security

- Each file has owner and group
- Permissions set by owner
 - Read, write, execute
 - Owner, group, other
 - Represented by vector of four octal values
- Only owner, root can change permissions
 - This privilege cannot be delegated or shared
- Setuid bits – More will come



In Unix different meaning for different objects

- Files:
 - Read: read from a file
 - Write: write to a file
 - eXecute: execute a (program) file
- Directories:
 - Read: list directory contents
 - Write: create, rename, or delete files
 - eXecute: access file contents and metadata and modify file list

eXecute: not as simple as it seems

- eXecute on a directory
 - write + execute permissions on dir, you can delete/rename files even if you do not have write permissions on them
 - execute on dir + write on file, you cannot delete the file!
 - Why?
- sticky bit
 - when this bit is set, user can only delete files if owner (useful to grant access to multiple users without letting them delete each other's file)

eXecute: not as simple as it seems

- eXecute on a directory
 - write + execute permissions on dir, you can delete/rename files even if you do not have write permissions on them
 - execute on dir + write on file, you cannot delete the file!
 - Remember what is directory
- sticky bit
 - when this bit is set, user can only delete files if owner (useful to grant access to multiple users without letting them delete each other's file)

eXecute: not as simple as it seems

- eXecute on a directory
 - write + execute permissions on dir, you can delete/rename files even if you do not have write permissions on them
 - execute on dir + write on file, you cannot delete the file!
- sticky bit
 - when this bit is set, user can only delete files if owner (useful to grant access to multiple users without letting them delete each other's file)
 - `chmod 1777 filename?`

eXecute: not as simple as it seems

- eXecute on a directory
 - write + execute permissions on dir, you can delete/rename files even if you do not have write permissions on them
 - execute on dir + write on file, you cannot delete the file!
- sticky bit
 - when this bit is set, user can only delete files if owner (useful to grant access to multiple users without letting them delete each other's file)
 - `chmod 1777 filename?` user sticky bit which allows only the user to have write access.

Owner in Unix

- Owner can have fewer privileges than other
 - So...
 - Owner gets access?
 - Owner does not?
- Prioritized resolution of differences
 - if user = owner then owner permission
else if user in group then group permission
else other permission

Access control lists (ACLs) in general

- More general ACLs do list per object what rights the listed subjects have to the object
- Easy answer to the question “who has what kind of access to this object”

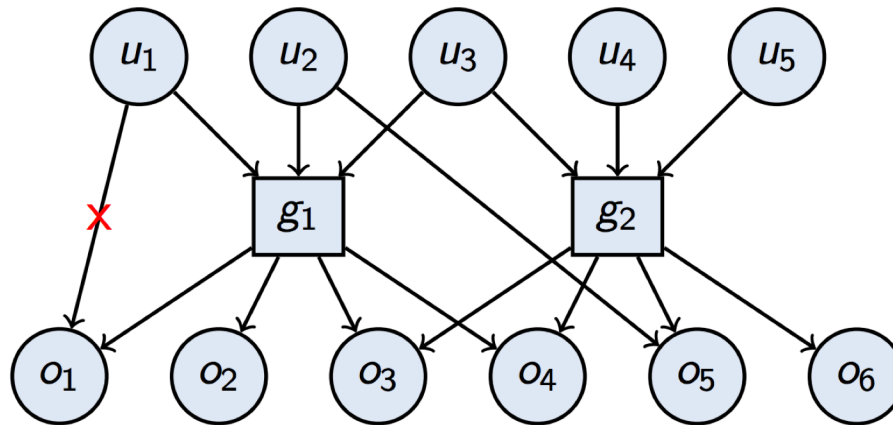
bill.doc	Bill:rw
edit.exe	Alice:rx, Bill:rx
tetris.exe	Alice:rx, Bill:rw

Issues in ACL?

Issues in ACL?

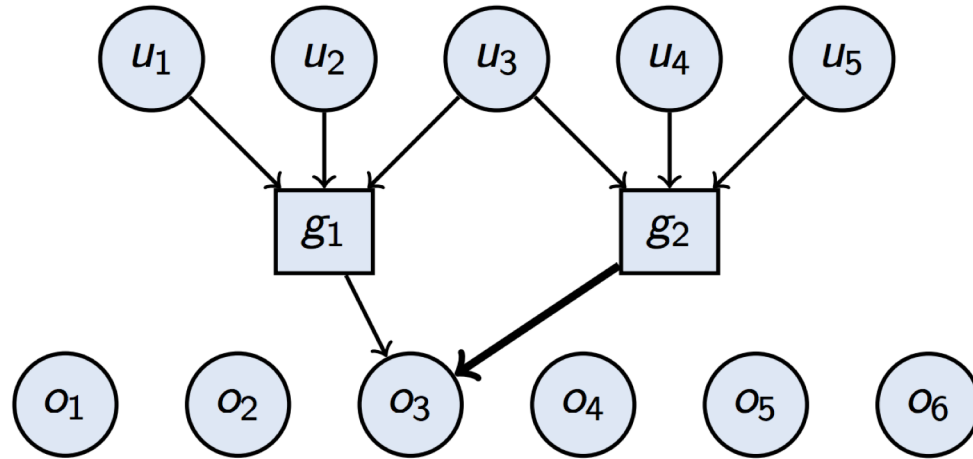
- May be inefficient:
 - checking rights for one user may require searching a long list
 - Revoking permissions for one user needs searching through all ACLs
- Popular despite shortcomings
 - Simple to implement
 - Natural in systems with discretionary access control
 - Unix-based systems and Windows (although the latter is more complicated)

Intermediate levels in ACLs



- Use groups to shorten lists
- Individual permissions can still be used
- Negative permissions can be used to create a smaller group out of an existing large one
- But now you need a rule for the case when someone (through groups) appears twice with different rights!

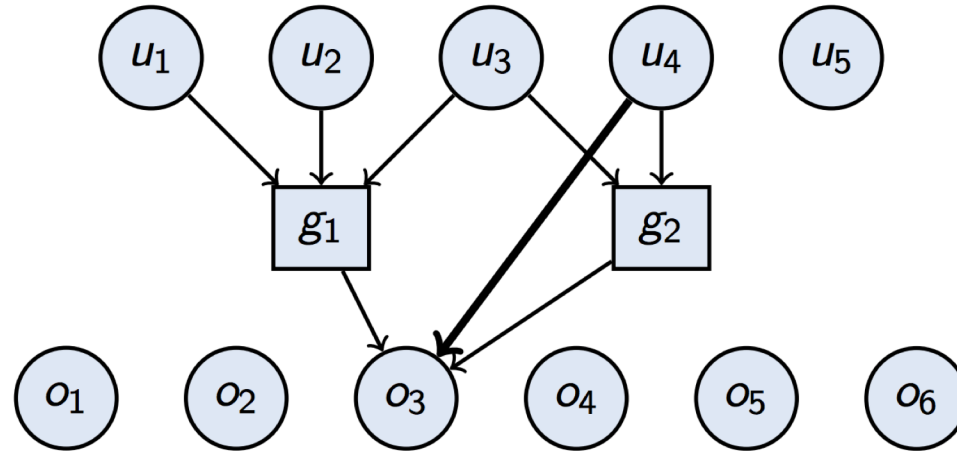
Intermediate levels in ACLs



g_1	u_1, u_2, u_3
g_2	u_3, u_4, u_5
o_3	$g_1:r \ g_2:rw$
$\vdots$	

- Two possible rules to resolve conflicts:
 - First match
 - Any match
- If u_3 wants to write to o_3 ,
 - the “first match” rule **denies this**
 - the “any match” rule **allows for this**

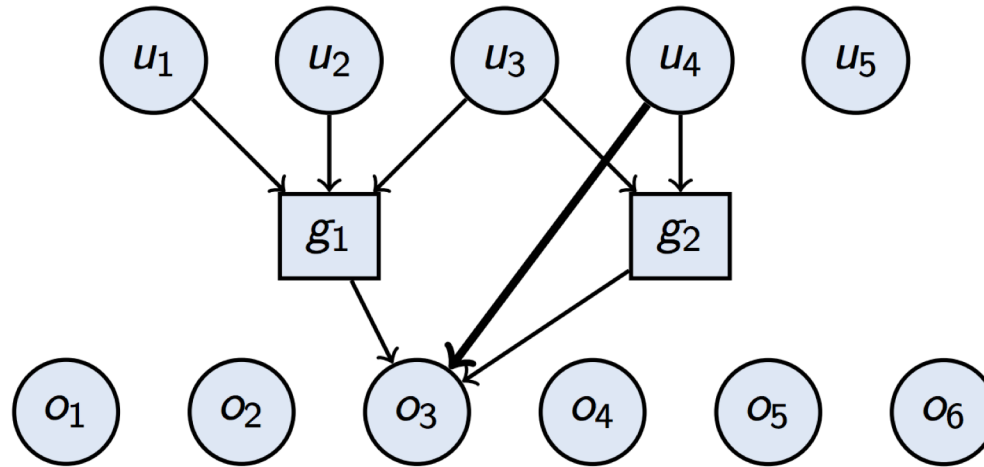
Any match



g_1	u_1, u_2, u_3
g_2	u_3, u_4
o_3	$g_1:r \ g_2:r$
	$\dots \ u_4:rw$
$\vdots$	

- If the list for o_3 is long, then it takes a long time to check if u_4 has write access
- Efficiency depends on ACL order
- For o_3 removal of $g_2:r$ does not change anything

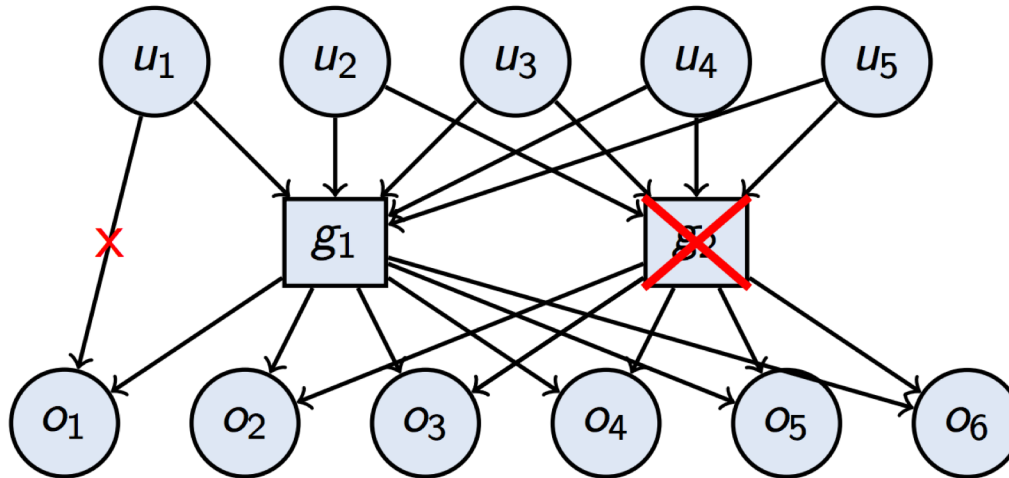
First match



g_1	u_1, u_2, u_3
g_2	u_3, u_4
o_3	$g_1:r \ g_2:r$
	$\dots \ u_4:rw$
$\vdots$	

- Finding u_4 's access rights is fast even though the list is long
- But here u_4 does not have write access to o_3
- Placing u_4 first slows down the search for other users
- For o_3 removal of $g_2:r$ gives u_4 write access

Negative permissions in ACLs



g_1	$u_1, u_2, u_3,$ u_4, u_5
o_1	$u_1:\text{None } g_1:r$
$\vdots$	

- Negative access rights can be used for the exceptional case
- This makes the ACL structure more efficient
- First match: negative must be first
- Any match: negative wins

Capabilities

- Lists per subject what right the subject has to what objects
- Easy answer to the question “what kind of access does this subject have to what objects”

Alice	<code>edit.exe:rx, tetris.exe:rx</code>
Bill	<code>bill.doc:rw, edit.exe:rx, tetris.exe:rwx</code>

Capabilities: Issues?

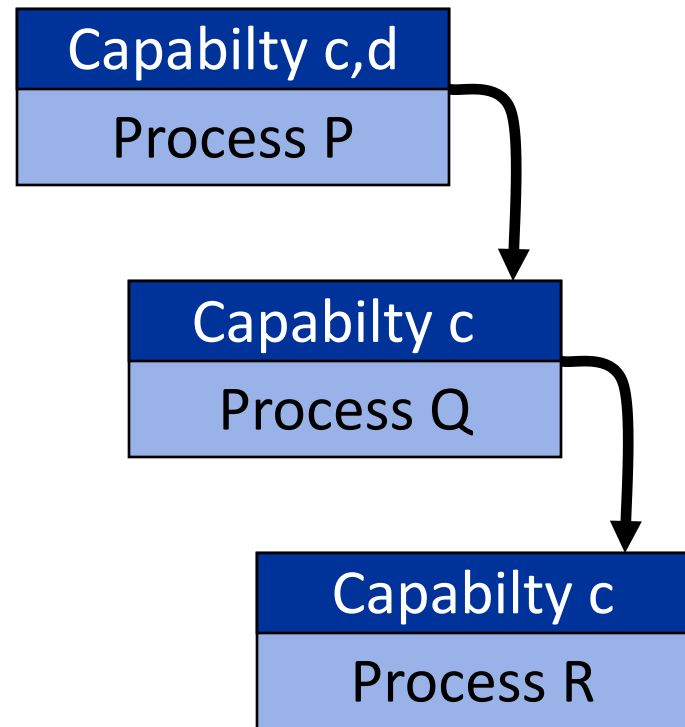
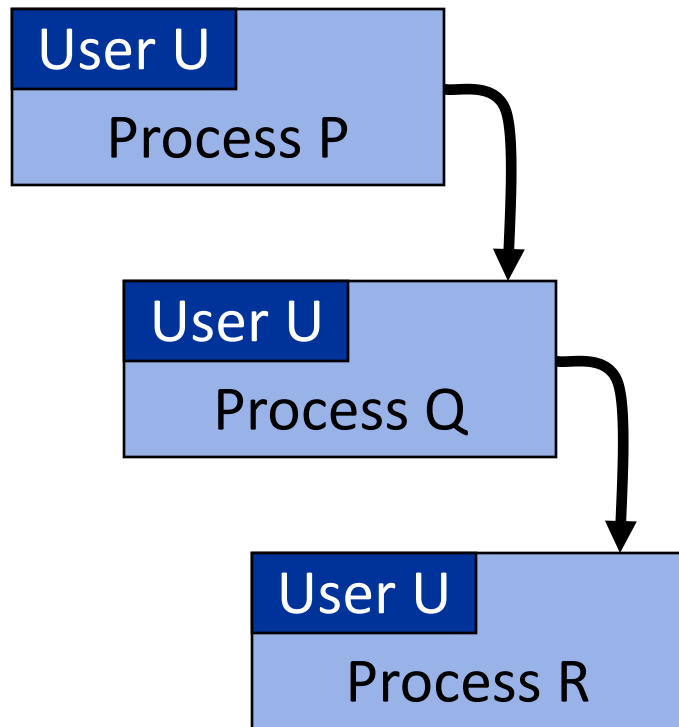
Capabilities: Issues

- May also be inefficient, since determining who has what rights for one object will require searching all users
- Here, principals (users) will pass on rights to subjects (process)
 - To be able to revoke permissions a principal needs to keep track of this
- Works better in a distributed setting, since roaming users have their permissions close at hand (but you need to implement this securely)

ACL vs Capabilities

- Access control list
 - Associate list with each object
 - Check user/group against list
 - Relies on authentication: need to know user
- Capabilities
 - Capability is unforgeable [ticket](#)
 - Random bit sequence, or managed by OS
 - Can be passed from one process to another
 - Reference monitor checks ticket
 - Does not need to know identity of user/process

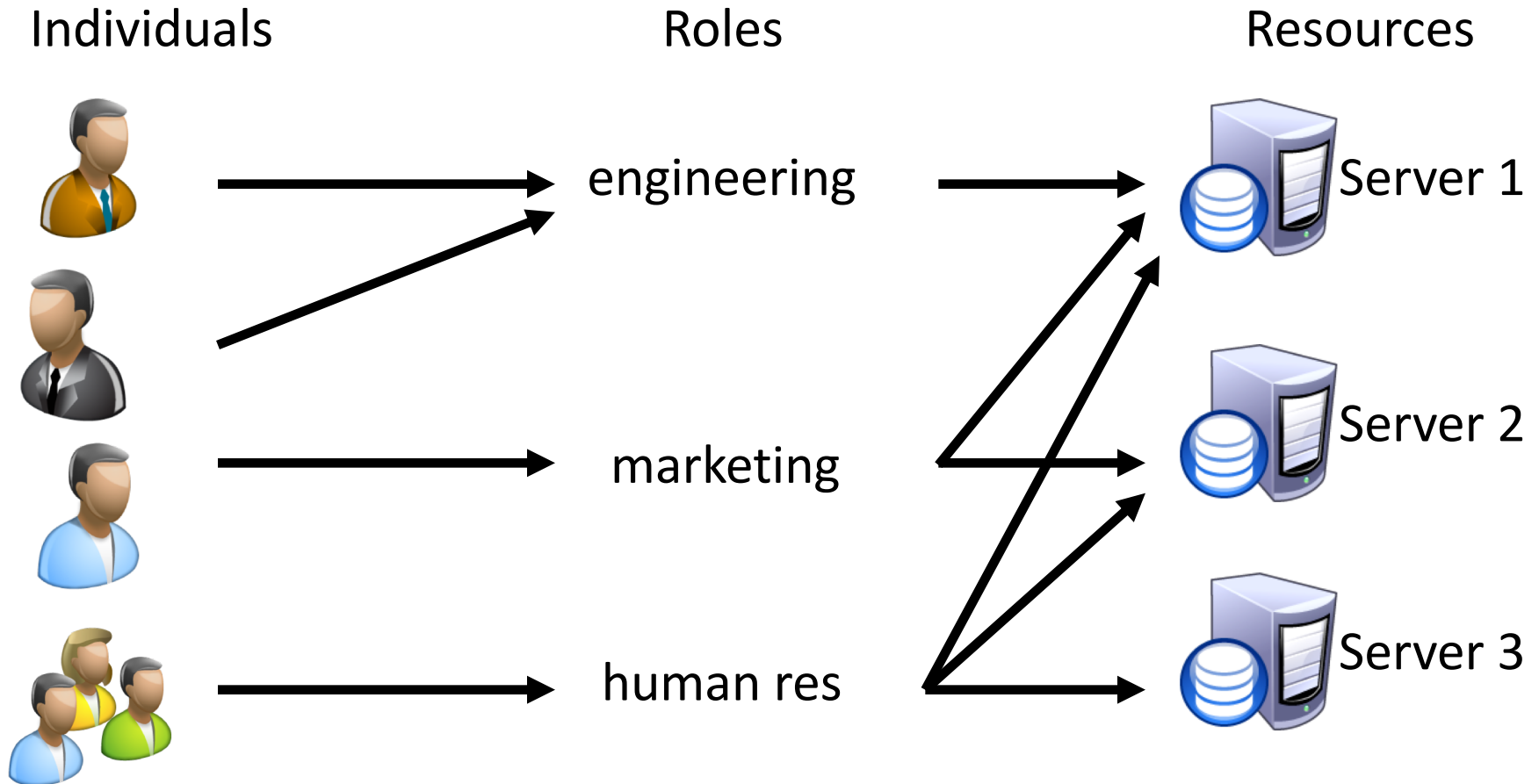
ACL vs Capabilities



ACL vs Capabilities

- Delegation
 - Capabilities, *easy*: process can pass capability at run time
 - ACL, *harder*: try to get owner to add permission to list?
 - More common: let other process act under current user
- Revocation
 - ACL, *easy*: remove user or group from list
 - Capabilities, *harder*: try to get capability back from process?
 - and what is the process has delegated the capability?
 - Possible in some systems if appropriate bookkeeping
 - OS knows which data is capability
 - If capability is used for multiple resources, have to revoke all or none ...

Role-Based Access Control (RBAC)



Advantage: user's change more frequently than roles

Role-Based Access Control (RBAC)

- A **role** is a collection of procedures, that users that have a role can execute
- A **procedure** is a more complicated operation (not merely read or write), for example a fund transfer between bank accounts
- A **data type** is associated to the objects, and to each data type, only specific procedures apply
- Frequently a user needs to take an explicit action to activate a desired role
- Roles often have a hierarchical structure
 - Example: infosec has Teacher, TA, Tutor, Student
- This is not always the hierarchy of the organization

Outline

- Access control
- In practice: Unix access control
- Breaking access: memory attacks
 - Exploiting overflow
 - Return oriented programming (ROP)

Unix access Control

From theory to practice

- The reality is way more complicated
 - access control policies cannot be always specified as one would like:
security requires [good programming practices](#)
 - the fundamental assumption underlying security models is that the reference monitor cannot be bypassed:
security requires protection against [hijacking attacks!](#)
 - Plan for the next lectures: first learn how to attack and then how to build a secure monitor

Unix, access rights to programs rather than users...

- You cannot always grant users direct access to files
- Solution is to grant program the access

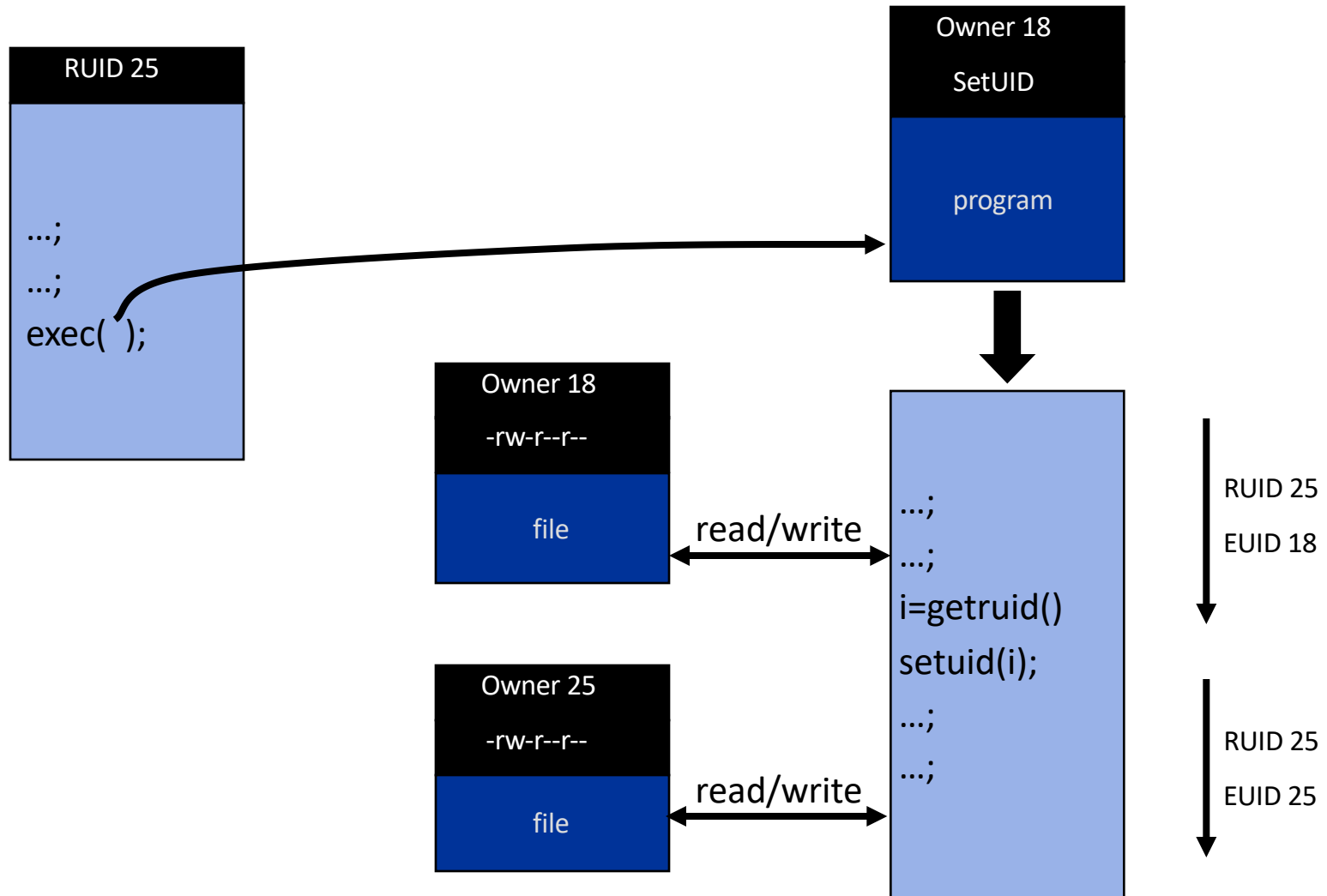
Unix, access rights to programs rather than users...

- In Unix, each process has three Ids (+ more under Linux)
 - Real user ID (RUID)
 - same as the user ID of parent (unless changed)
 - used to determine which user started the process
 - Effective user ID (EUID)
 - determined by setuid bit on the file being executed, or sys call: if set, EUID is the executable's owner ID
 - determines the permissions for process
 - file access and port binding
 - Saved user ID (SUID)
 - So previous EUID can be restored
- Real group ID, effective group ID, used similarly

Process Operations and IDs

- Root
 - ID=0 for superuser root; can access any file
- Fork and Exec
 - Inherit three IDs, except exec of file with setuid bit
- Setuid system calls
 - seteuid(newid) can set EUID to
 - Real ID or saved ID, regardless of current EUID
 - Any ID, if EUID=0
- Details are actually more complicated
 - Several different calls: setuid, seteuid, setreuid

Example



Setuid might be evil

- Careful with Setuid!
 - Can do anything that owner of file is allowed to do
 - Be sure not to
 - Take action for untrusted user
 - Return secret data to untrusted user

NOTE: anything possible if root; no middle ground between user and root

Setuid programming

- Be Careful!
 - Root can do anything; don't get tricked
- Be sure not to
 - Take action for untrusted user
 - Return secret data to untrusted user
 - Principle of least privilege – change EUID when root privileges no longer needed
- Setuid scripts
 - This is a bad idea
 - Historically, race conditions
 - Begin executing setuid program; change contents of program before it loads and is executed
 - model Unix systems ignore setuid bit on scripts

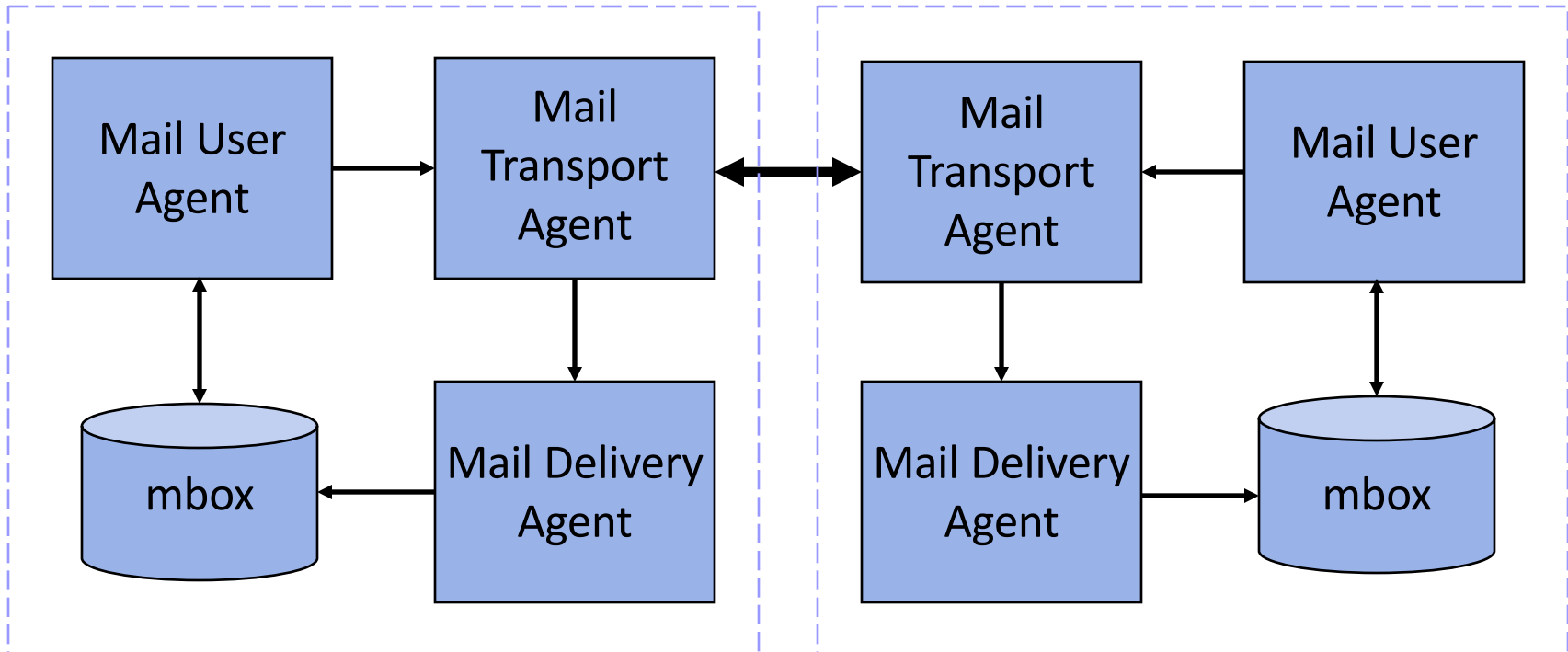
Unix: The good and bad

- Good things
 - Some protection from most users
 - Flexible enough to make things possible
- Main bad thing
 - Too tempting to use root privileges
 - No way to assume some root privileges without all root privileges

Example: Mail Transport Agents

- Sendmail
 - Complicated system, many past vulnerabilities
 - Sendmail runs as root
 - Root privilege needed to bind port 25
 - No longer needed after port bind established
 - But most systems keep running as root
 - Root privileges needed later to write to user mailboxes
 - Situation improved over the years, incorporating a certain degree of [privilege separation](#)
- Qmail
 - Simpler system designed with security in mind
 - Qmail was written by Dan Bernstein, starting 1995
 - \$500 reward for successful attack, no one has collected

Simplified Mail Transactions

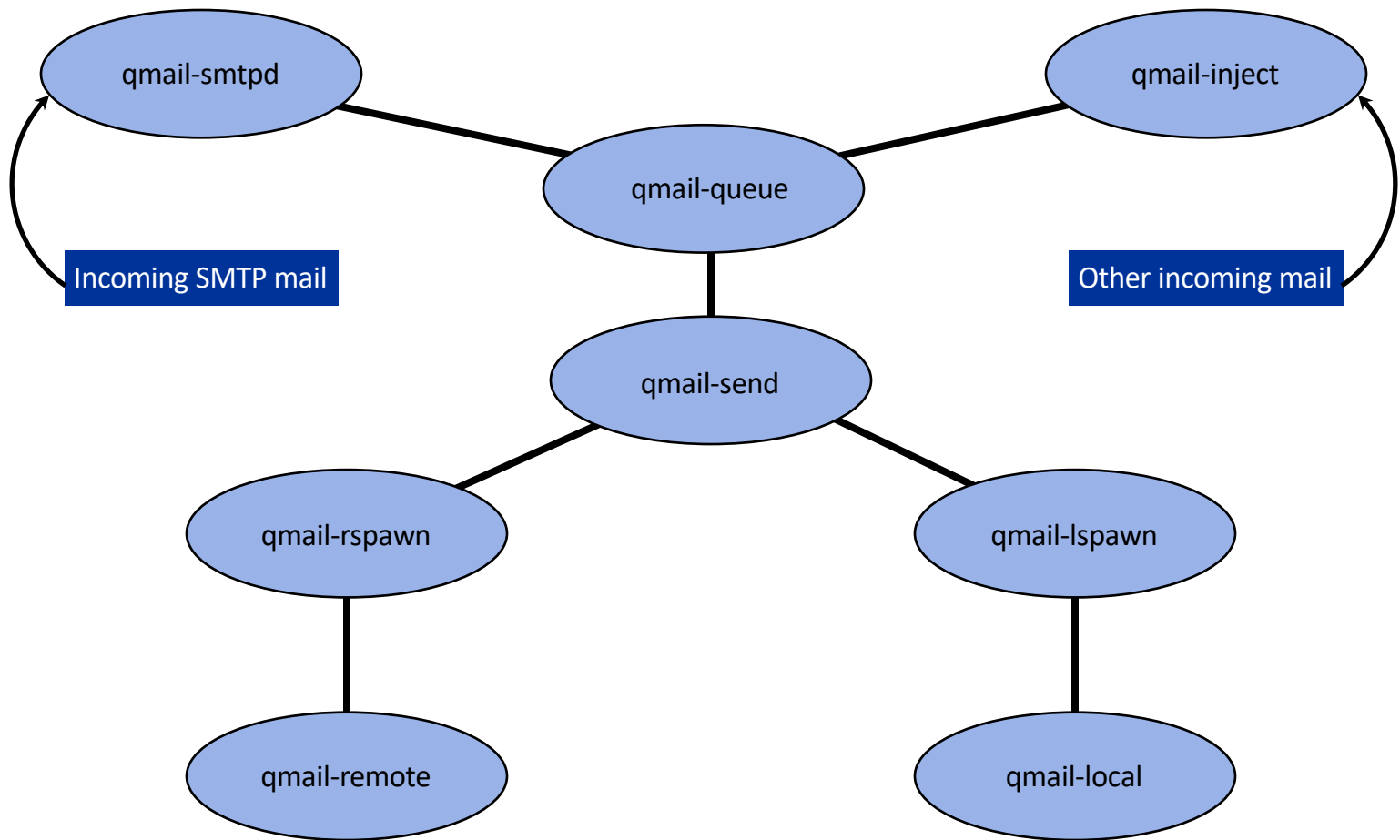


- Message composed using an MUA
- MUA gives message to MTA for delivery
 - If local, the MTA gives it to the local MDA
 - If remote, transfer to another MTA

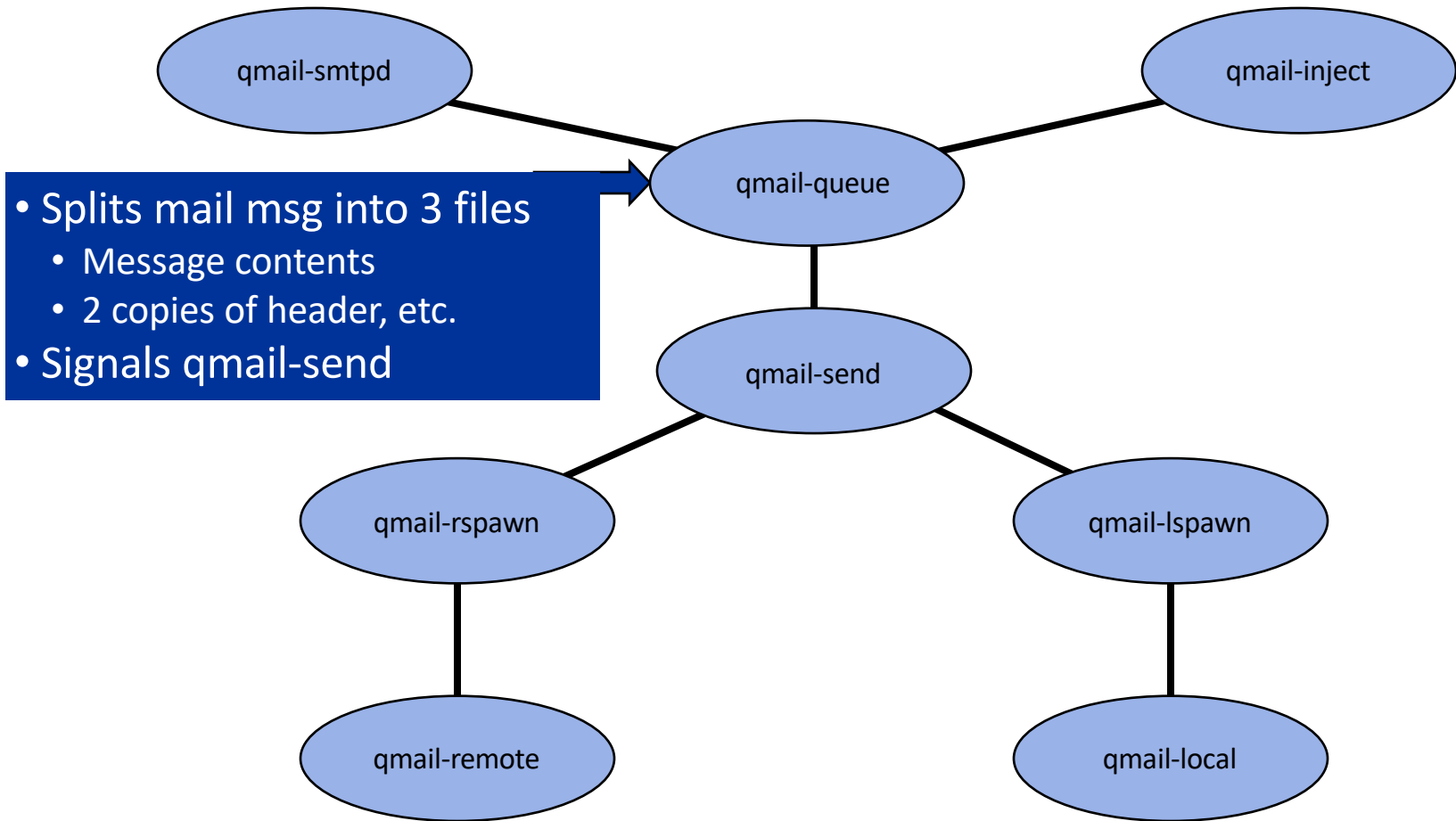
Qmail design

- Least privilege
 - Each module uses least privileges necessary
 - Only one setuid program
 - setuid to one of the other qmail user IDs, not root
 - No setuid root binaries
 - Only one run as root
 - Spawns the local delivery program under the UID and GID of the user being delivered to
 - No delivery to root
 - Always changes effective uid to recipient before running user-specified program
- Other secure coding ideas

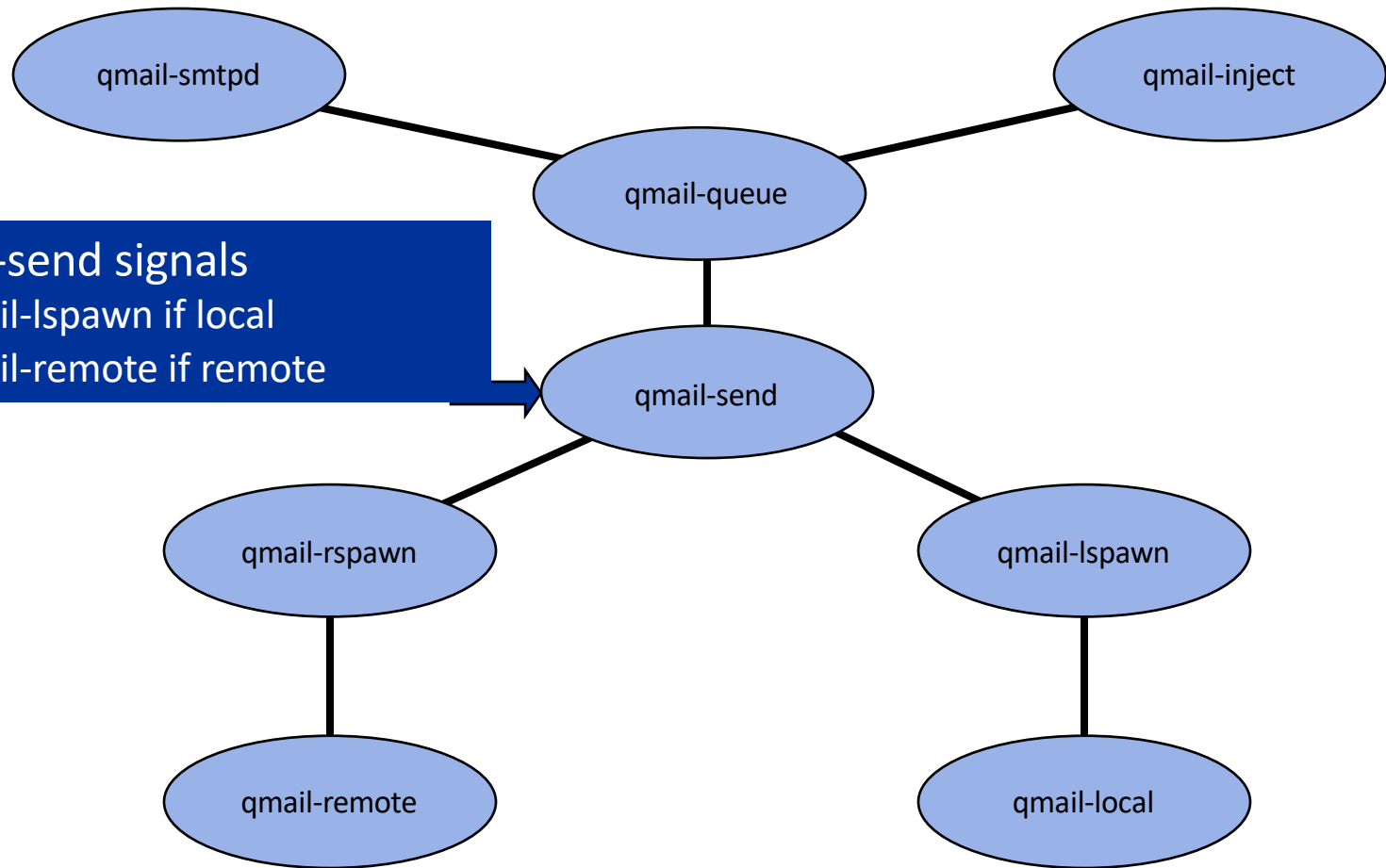
Structure of qmail



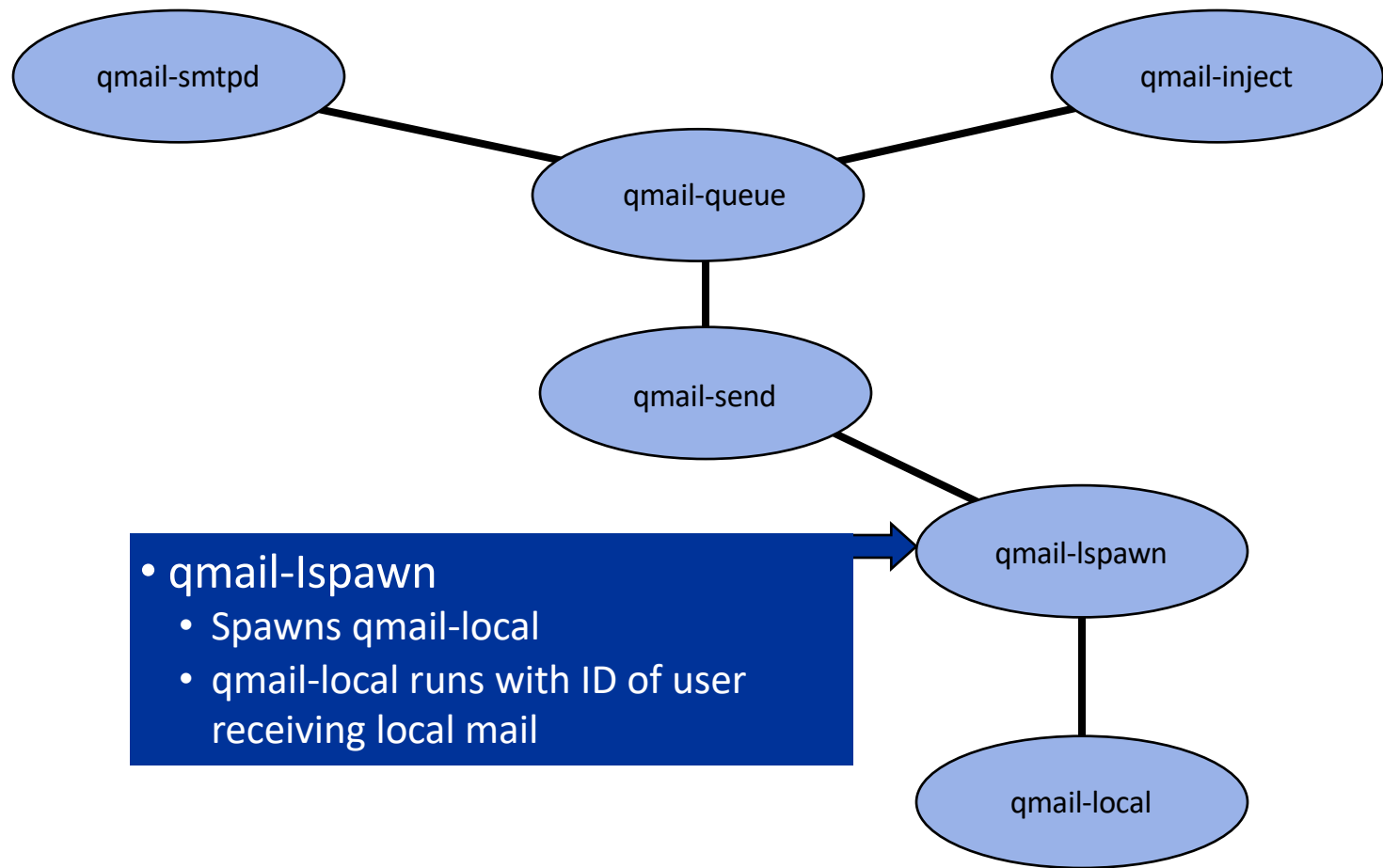
Structure of qmail



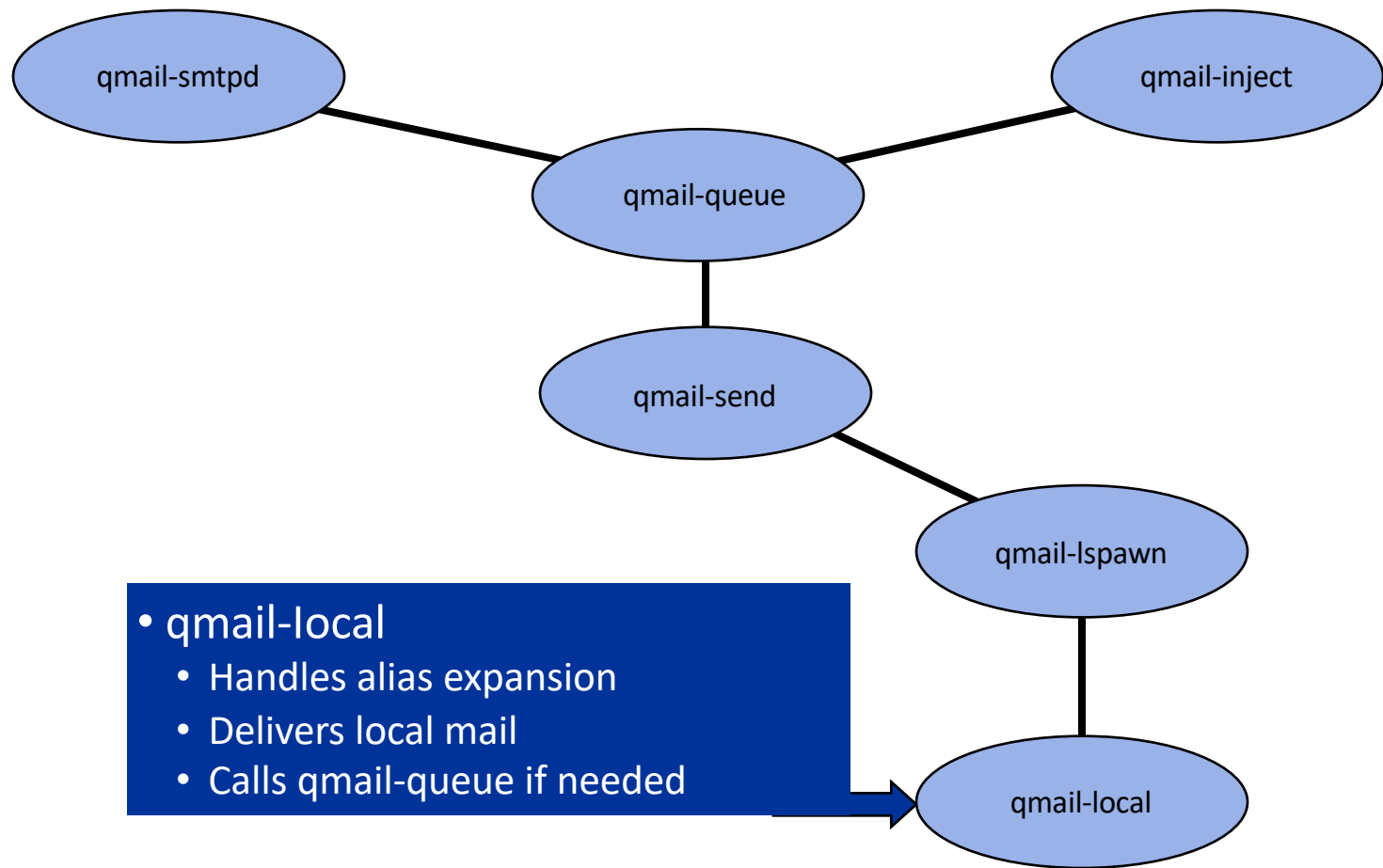
Structure of qmail



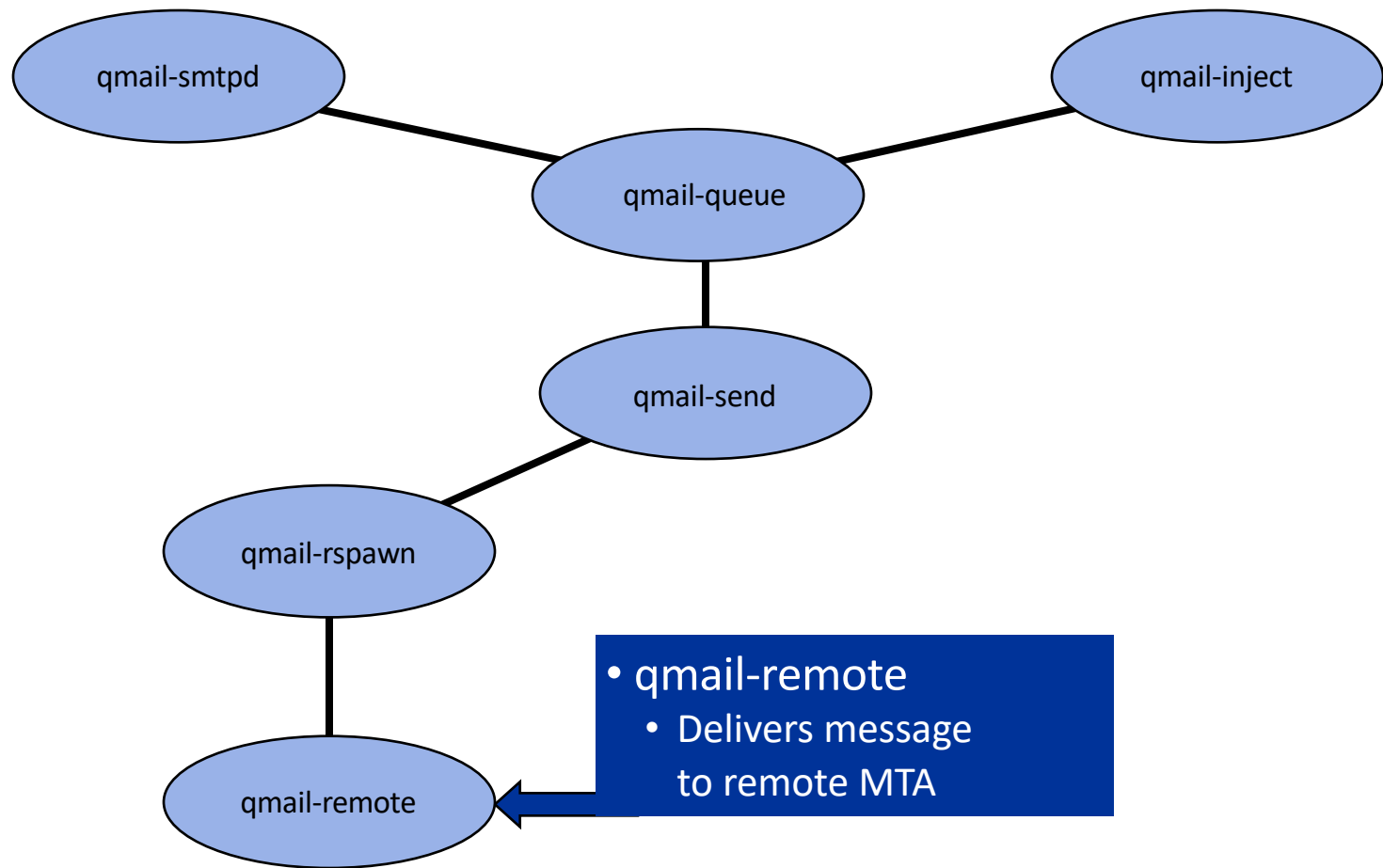
Structure of qmail



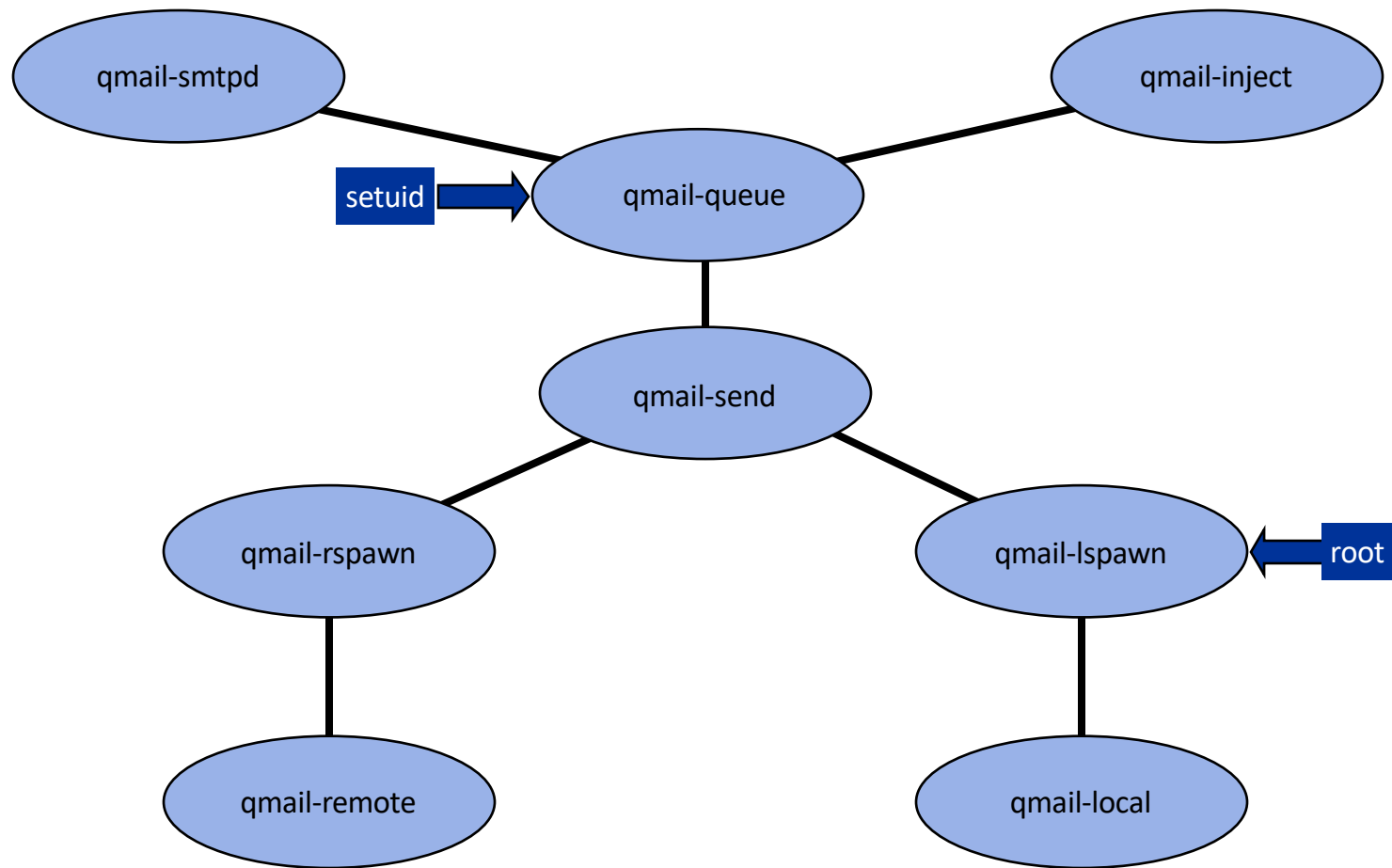
Structure of qmail



Structure of qmail

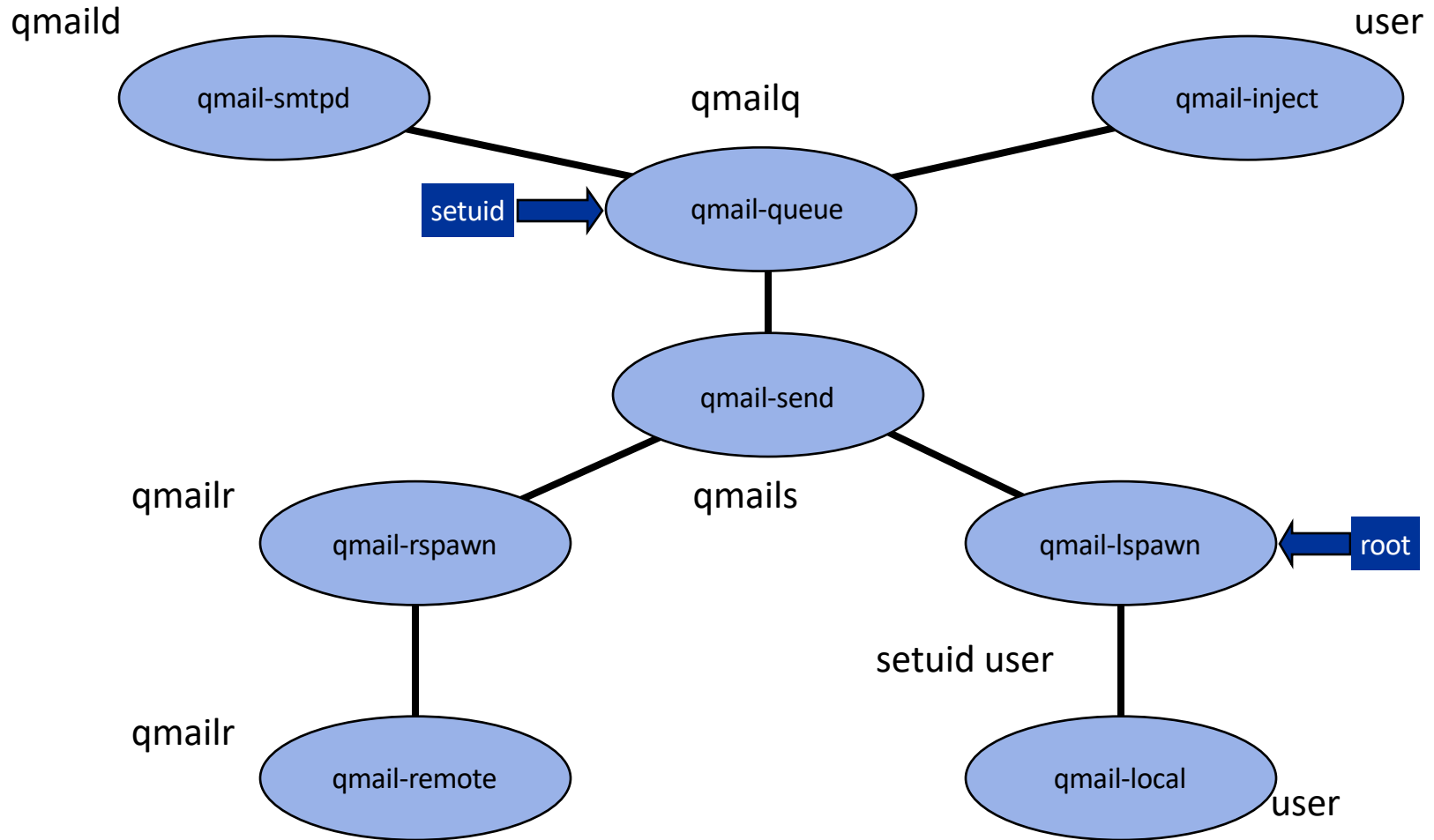


Least privilege



UIDs

<http://cr.yp.to/qmail/guarantee.html>



qmailq: user who is allowed to read/write mail queue

Principles, sendmail vs qmail

- Do as little as possible in setuid programs
 - Statistically, of 20 sendmail security holes, 11 worked only because the entire sendmail system is setuid
 - Only qmail-queue is setuid
 - Its only function is add a new message to the queue
- Do as little as possible as root
 - The entire sendmail system runs as root
 - Operating system protection has no effect
 - Only qmail-start and qmail-lspawn run as root.

Principles of secure design - summary

- Compartmentalization
 - Principle of least privilege
 - Minimize trust relationships
- Defense in depth
 - Use more than one security mechanism
 - Secure the weakest link
 - Fail securely
- Keep it simple! And Consult experts

Outline

- Access control
- In practice: Unix access control
- Breaking access: memory attacks
 - Exploiting overflow
 - Return oriented programming (ROP)